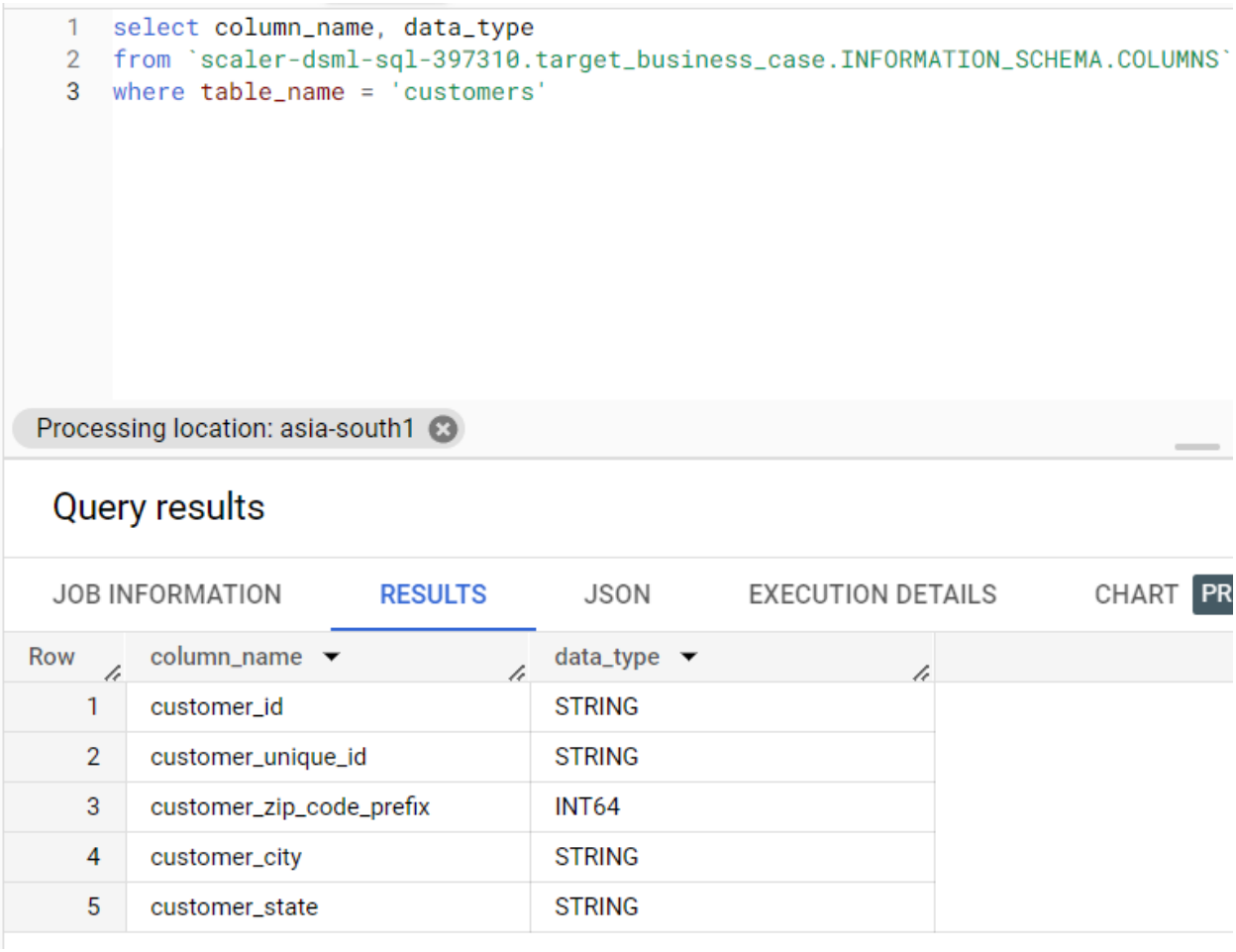
Business Case : Target SQL

Q1. Data type of all columns in the "customers" table.

```
select column_name, data_type
from `scaler-dsml-sql-397310.target_business_case.INFORMATION_SCHEMA.COLUMNS`
where table_name = 'customers'
```

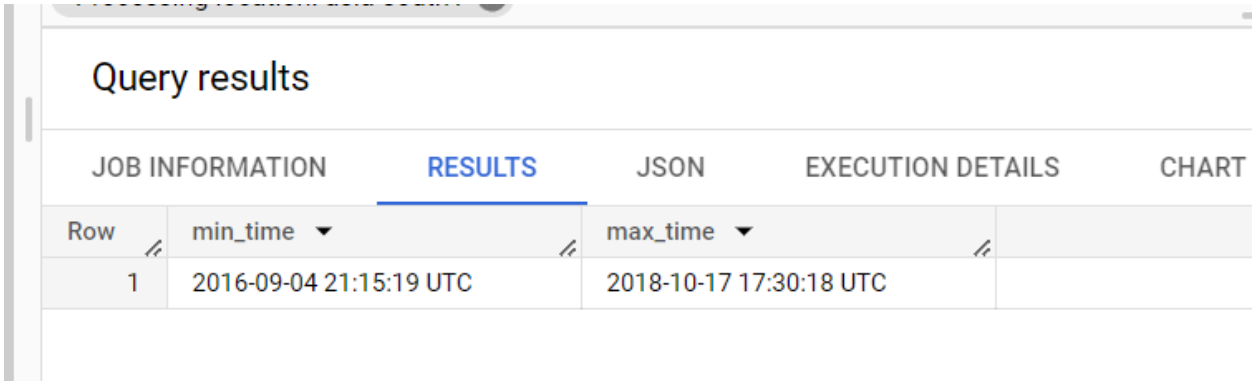


Row	column_name	data_type
1	customer_id	STRING
2	customer_unique_id	STRING
3	customer_zip_code_prefix	INT64
4	customer_city	STRING
5	customer_state	STRING

Insights - As you can see, except customer zip code which is in a 64-bit integer datatype all the other columns are in string datatype.

Q2. Get the time range between which the orders were placed.

```
select min(order_purchase_timestamp) as min_time, max(order_purchase_timestamp) as
max_time from `target_business_case.orders`;
```



Insight - As seen above maximum time taken by customer to place an order is 17:30 in 2018 and minimum time is 21 :15 in 2016

Q3. Count the Cities & States of customers who ordered during the given period.

```
select customer_state, count(distinct customer_state) as statecount,
count(customer_city) as citycount from `target_business_case.customers`
group by 1
```

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS

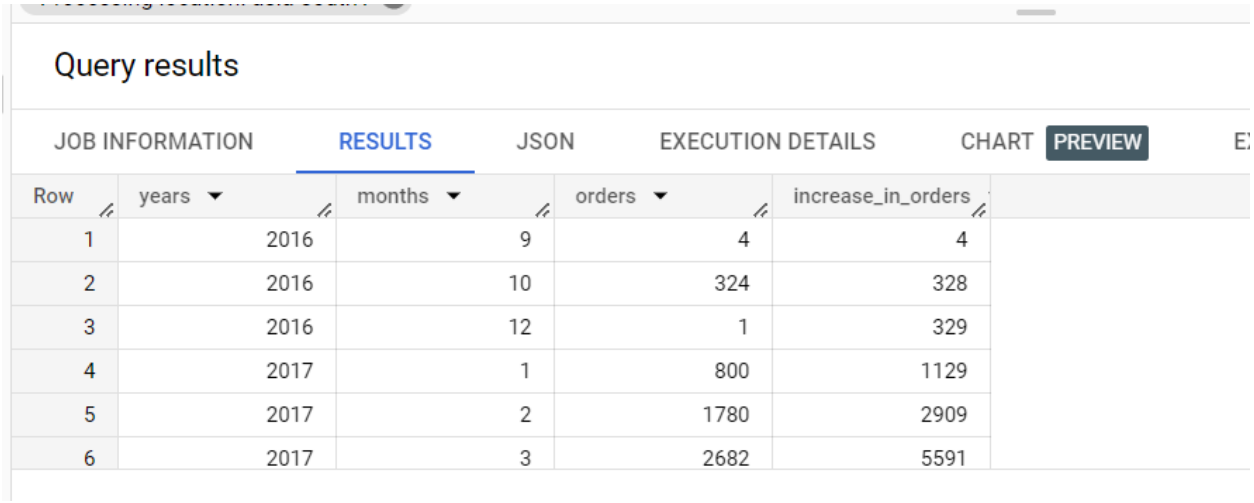
Row	customer_state	statecount	citycount
1	RN	1	485
2	CE	1	1336
3	RS	1	5466
4	SC	1	3637
5	SP	1	41746

Insight - As seen in the above query I have used distinct in state count because they are many cities for a particular state. And also if you look into the query results, maximum number of our customer are from SP (São Paulo) and minimum number are from RR (Roraima).

Q4. Is there a growing trend in the no. of orders placed over the past years?

```
with base as (
      select extract(year FROM order_purchase_timestamp) as years, extract(month FROM
order_purchase_timestamp) as months, count(order_id)
      as orders from `target_business_case.orders`
      group by 1,2
      order by 1,2
)

select years, months, orders, sum(orders) over(order by years, months) as
increase_in_orders from base
```



Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | CHART | PREVIEW

Row	years	months	orders	increase_in_orders
1	2016	9	4	4
2	2016	10	324	328
3	2016	12	1	329
4	2017	1	800	1129
5	2017	2	1780	2909
6	2017	3	2682	5591

Insight - After you run this query you will see a significant amount of growth rate in orders count between 2016 till 2018 (as per the data).Especially on March 2018.

Q5. Can we see some kind of monthly seasonality in terms of the no. of orders being placed?

```
 with base as (select extract(year FROM order_purchase_timestamp) as years,
extract(month FROM order_purchase_timestamp) as months, FORMAT_DATETIME('%B',
order_purchase_timestamp) as month_name, count(order_id)
      as orders from `target_business_case.orders`
      group by 1,2,3
      order by 1,2
 ),

base1 as (select years, months, month_name, orders, avg(orders) over(partition by
months) as avg_order from base order by 1,2)

select distinct months, month_name, avg_order from base1 order by 1
```

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | CHART

Row	months	month_name	avg_order
1	1	January	4034.5
2	2	February	4254.0
3	3	March	4946.5
4	4	April	4671.5
5	5	May	5286.5
6	6	June	4706.0

Insight - Yes, as per the data most customers place order in the month of November and on the other hand very less order are placed in the month September.

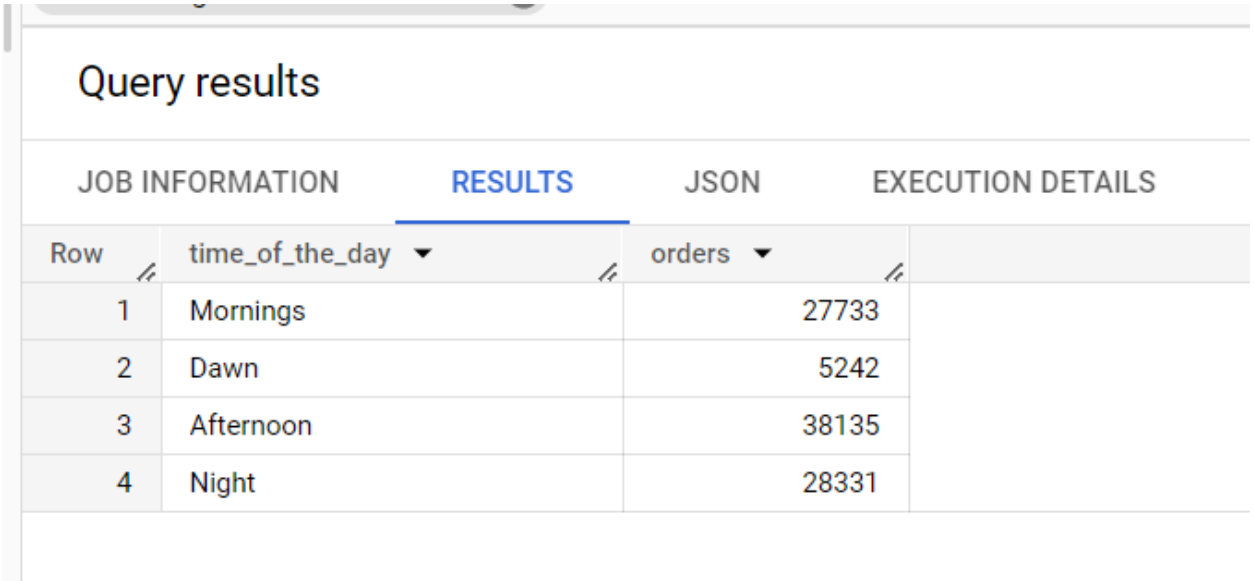
Q6. During what time of the day, do the Brazilian customers mostly place their orders? (Dawn, Morning, Afternoon or Night)

- 0-6 hrs : Dawn
- 7-12 hrs : Mornings
- 13-18 hrs : Afternoon
- 19-23 hrs : Night

```
with base as (select extract(hour from order_purchase_timestamp) as hours,
              count(order_id) as orders
              from `target_business_case.orders`
              group by 1
),

time_of_the_day_table as (select *, case when hours between 0 and 6 then 'Dawn'
                    when hours between 7 and 12 then 'Mornings'
                    when hours between 13 and 18 then 'Afternoon'
                    when hours between 19 and 23 then 'Night'
                    end as time_of_the_day,
                from base )

select time_of_the_day, sum(orders) as orders from time_of_the_day_table group by 1
```



Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS

Row	time_of_the_day	orders
1	Mornings	27733
2	Dawn	5242
3	Afternoon	38135
4	Night	28331

Insight - As seen in the above query results most orders are placed during the Afternoon amd very less in the Dawn.

Q7. Get the month on month no. of orders placed in each state.

```
with base as (select c.customer_state, extract(month from o.order_purchase_timestamp)
as months, count(order_id) as orders from `target_business_case.orders` as o
inner join `target_business_case.customers` as c
on o.customer_id = c.customer_id
group by 1, 2
order by 1,2),


base1 as (select customer_state, months, orders, lag(orders) over(order by
customer_state, months) as lag_orders from base order by 1,2)


select customer_state, months, orders, round(100 * ((orders/lag_orders) -1),0) as
perc_month_on_month from base1 order by 1,2
```

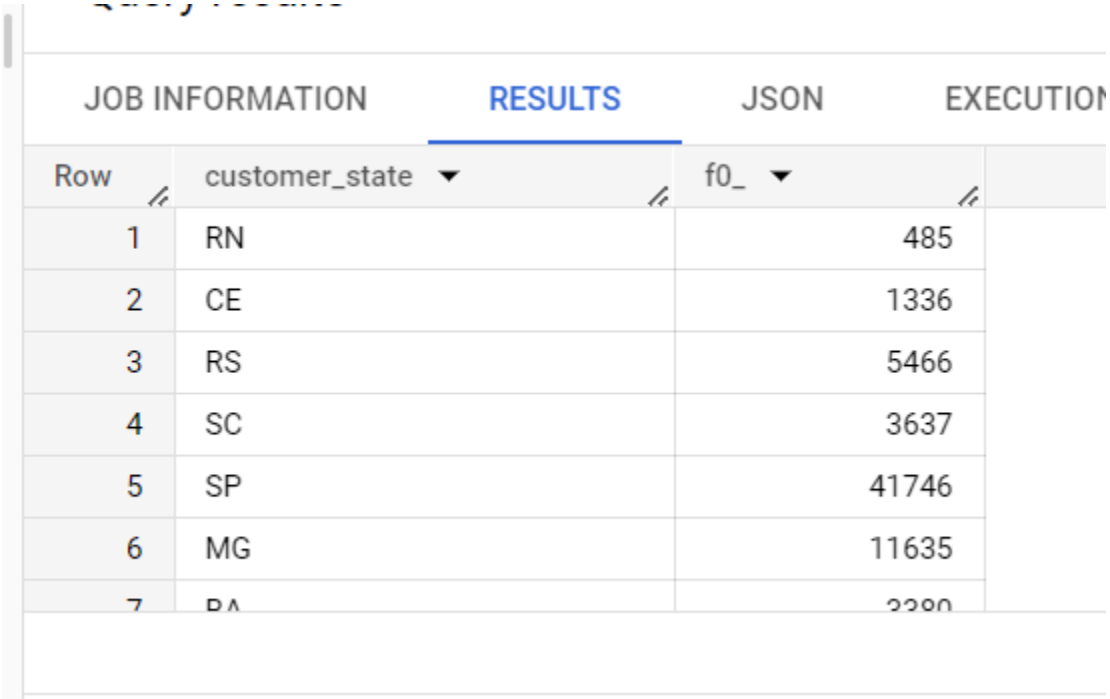
JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | CHART PREVIEW | EX

Row	customer_state	months	orders	perc_month_on_mon
1	AC	1	8	null
2	AC	2	6	-25.0
3	AC	3	4	-33.0
4	AC	4	9	125.0
5	AC	5	10	11.0
6	AC	6	7	-30.0
7	AC	7	9	28.6

Insight - through this query result you will get to the month on month growth rate in percentile as well as in no. of orders format, which let you know most no. of orders placed in a particular month in specific state

Q8. How are the customers distributed across all the states?

```
select customer_state, count(distinct customer_id) from
`target_business_case.customers` group by 1
```



JOB INFORMATION | RESULTS | JSON | EXECUTION

Row	customer_state	f0_
1	RN	485
2	CE	1336
3	RS	5466
4	SC	3637
5	SP	41746
6	MG	11635
7	RA	2380

Insight - Through the results we can see most number of customers are from SP (São Paulo) which is the industrial hub and also holds 33.9% of GDP. On other hand is Roraima with less of customers and this state is know for it's amazon forest which clearly states that it bounds to have less customers due to less development and rough roads.

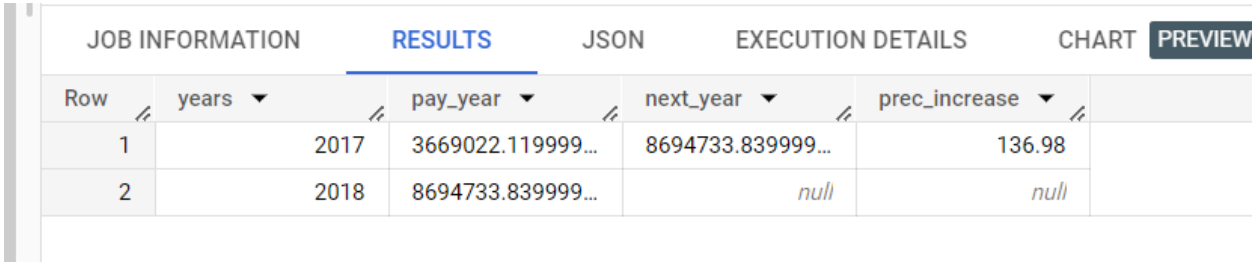
Q9. Get the % increase in the cost of orders from year 2017 to 2018 (include months between Jan to Aug only).

```
with base as (select extract(year from o.order_purchase_timestamp) as years,
extract(month from o.order_purchase_timestamp) as months, sum(p.payment_value) as
payments from `target_business_case.orders` as o
inner join `target_business_case.payments`as p
on o.order_id = p.order_id
where extract(month from o.order_purchase_timestamp) between 1 and 8
group by 1,2
order by 1,2
),

base1 as (select years, sum(payments) as pay_year from base group by 1 order by 1),

lag as (select *, lag(pay_year, 1) over(order by 1) as next_year from base1)

select *, round(100 * (next_year - pay_year)/ pay_year,2) as prec_increase from lag
order by 1
```



JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | CHART | PREVIEW

Row	years	pay_year	next_year	prec_increase
1	2017	3669022.119999...	8694733.839999...	136.98
2	2018	8694733.839999...	null	null

Insight - There is a 136.98 percent increase in cost of orders from 2017 to 2018.

Q10. Calculate the Total & Average value of order price for each state.

```
select c.customer_state, round(sum(ot.price),2) as total_price, round(avg(ot.price),
2) as avg_price from `target_business_case.customers` as c
inner join `target_business_case.orders`as o
on c.customer_id = o.customer_id
inner join `target_business_case.order_items`as ot
on o.order_id = ot.order_id
group by 1
order by 1
```

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | CHAR

Row	customer_state	total_price	avg_price
1	AC	15982.95	173.73
2	AL	80314.81	180.89
3	AM	22356.84	135.5
4	AP	13474.3	164.32
5	BA	511349.99	134.6
6	CE	227254.71	153.76
7	DF	302603.94	125.77

Insight - Country with the maximum avg order price is PB (Paraíba) and country with maximum total order price is SP (São Paulo).

Q11. Calculate the Total & Average value of order freight for each state.

```
select c.customer_state, round(sum(ot.freight_value),2) as total_price,
round(avg(ot.freight_value), 2) as avg_price from `target_business_case.customers` as
c
inner join `target_business_case.orders`as o
on c.customer_id = o.customer_id
inner join `target_business_case.order_items`as ot
on o.order_id = ot.order_id
group by 1
order by 1
```

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | CHART | PRE

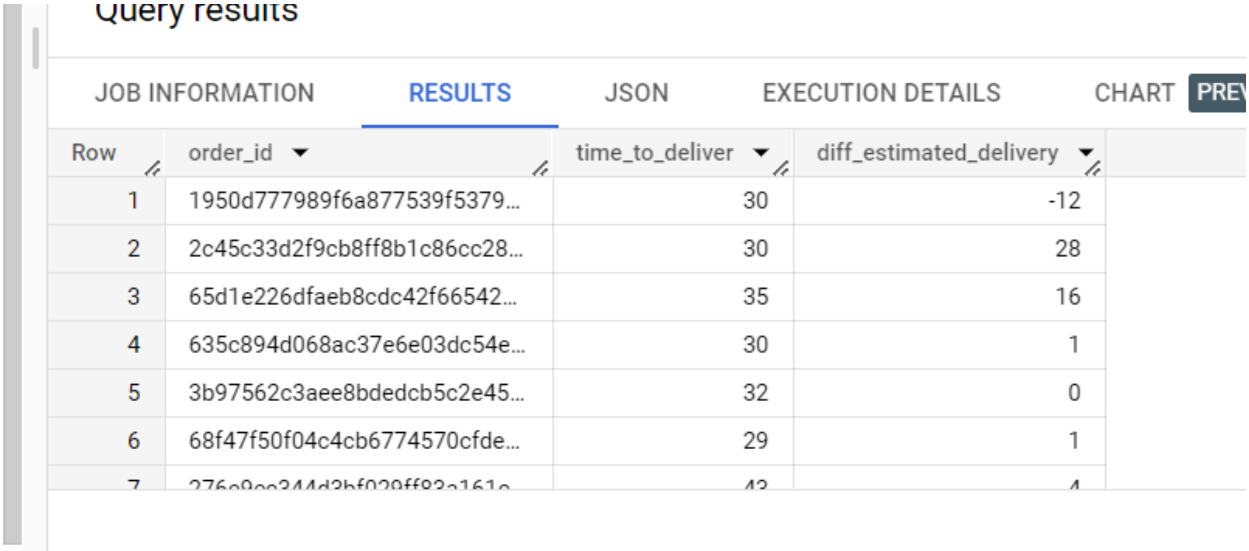
Row	customer_state	total_price	avg_price
1	AC	3686.75	40.07
2	AL	15914.59	35.84
3	AM	5478.89	33.21
4	AP	2788.5	34.01
5	BA	100156.68	26.36
6	CE	48351.59	32.71
7	DF	50625.5	21.04

PERSONAL HISTORY PROJECT HISTORY

Insight - Country with the most number of total freight values is SP (São Paulo) and country with the most number of avg freight values is RR(Roraima).

Q12. Find the no. of days taken to deliver each order from the order's purchase date as delivery time.
Also, calculate the difference (in days) between the estimated & actual delivery date of an order.
Do this in a single query.
Hint: You can calculate the delivery time and the difference between the estimated & actual delivery date using the given formula:
● time_to_deliver = order_delivered_customer_date - order_purchase_timestamp
● diff_estimated_delivery = order_estimated_delivery_date - Order_delivered_customer_date

```
select order_id, timestamp_diff(order_delivered_customer_date,
order_purchase_timestamp, day) as
time_to_deliver,timestamp_diff(order_estimated_delivery_date,
order_delivered_customer_date, day) as diff_estimated_delivery from
`target_business_case.orders` where order_delivered_customer_date is not null
```


Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | CHART | PREV

Row	order_id	time_to_deliver	diff_estimated_delivery
1	1950d777989f6a877539f5379...	30	-12
2	2c45c33d2f9cb8ff8b1c86cc28...	30	28
3	65d1e226dfaeb8cdc42f66542...	35	16
4	635c894d068ac37e6e03dc54e...	30	1
5	3b97562c3aee8bdedcb5c2e45...	32	0
6	68f47f50f04c4cb6774570cfde...	29	1
7	276e9ec344d3bf029ff83a161e...	43	4

Q13. Find out the top 5 states with the highest & lowest average freight value.

```
with lowest_value_table as (select c.customer_state as customer_state,
round(avg(ot.freight_value),2) as lowest_value from `target_business_case.customers`
as c
inner join `target_business_case.orders`as o
on c.customer_id = o.customer_id
inner join `target_business_case.order_items`as ot
on o.order_id = ot.order_id
group by 1
order by 2
limit 5
),

highest_value_table as (select c.customer_state as customer_state,
round(avg(ot.freight_value),2) as highest_value from `target_business_case.customers`
as c
inner join `target_business_case.orders`as o
on c.customer_id = o.customer_id
inner join `target_business_case.order_items`as ot
on o.order_id = ot.order_id
group by 1
order by 2 desc
limit 5
)

select customer_state, highest_value as avg_freight_value from highest_value_table as
h
union all
select customer_state, lowest_value from lowest_value_table as l
order by 2
```

JOB INFORMATION | RESULTS | JSON | EXECUTION DE

Row	customer_state	avg_freight_value
1	SP	15.15
2	PR	20.53
3	MG	20.63
4	RJ	20.96
5	DF	21.04
6	PI	39.15
7	AC	40.07
8	RO	41.07

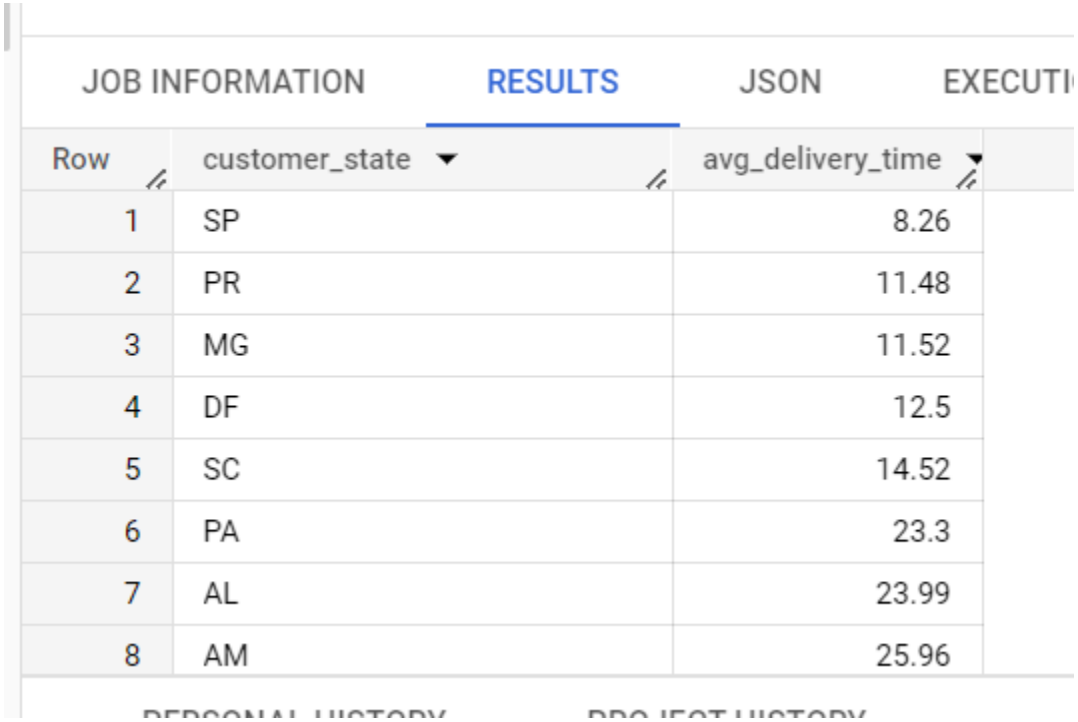
Q14. Find out the top 5 states with the highest & lowest average delivery time.

```
with lowest_value_table as (select c.customer_state as customer_state,
round(avg(timestamp_diff(order_delivered_customer_date, order_purchase_timestamp,
day)),2) as lowest_value from `target_business_case.customers` as c
inner join `target_business_case.orders`as o
on c.customer_id = o.customer_id
inner join `target_business_case.order_items`as ot
on o.order_id = ot.order_id
group by 1
order by 2
limit 5
),

highest_value_table as (select c.customer_state as customer_state,
round(avg(timestamp_diff(order_delivered_customer_date, order_purchase_timestamp,
day)),2) as highest_value from `target_business_case.customers` as c
inner join `target_business_case.orders`as o
on c.customer_id = o.customer_id
inner join `target_business_case.order_items`as ot
on o.order_id = ot.order_id
group by 1
```

```
order by 2 desc
limit 5
)

select customer_state, highest_value as avg_delivery_time from highest_value_table as
h
union all
select customer_state, lowest_value from lowest_value_table as l
order by 2
```



JOB INFORMATION | RESULTS | JSON | EXECUTI

Row	customer_state	avg_delivery_time
1	SP	8.26
2	PR	11.48
3	MG	11.52
4	DF	12.5
5	SC	14.52
6	PA	23.3
7	AL	23.99
8	AM	25.96

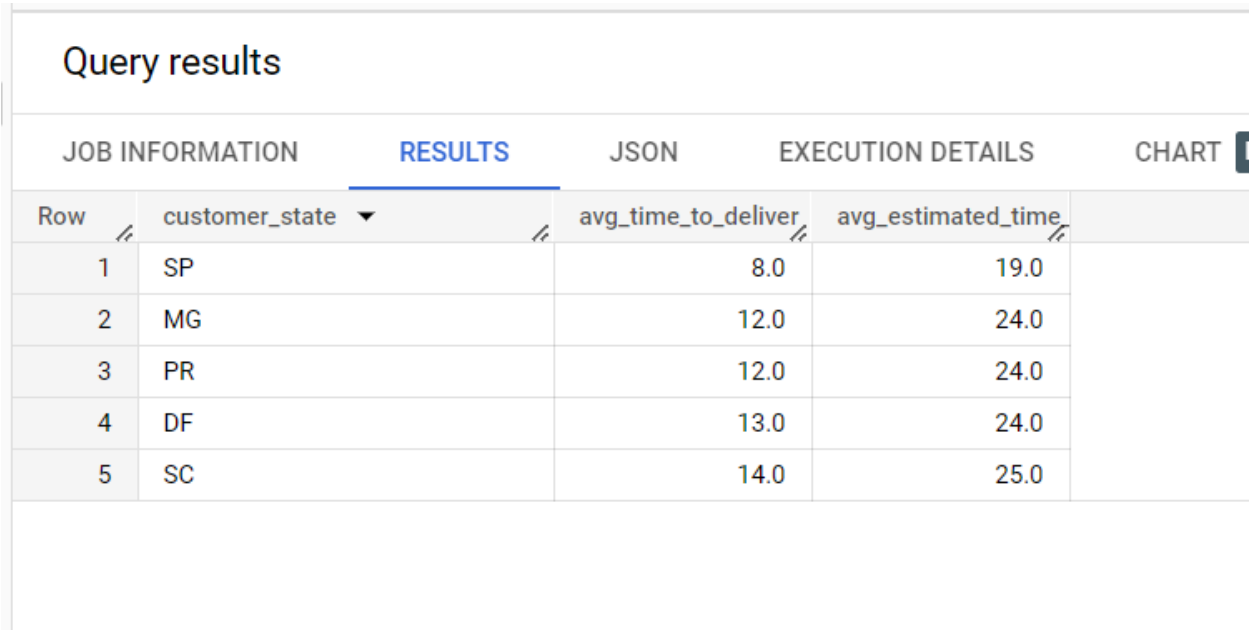
Q15. Find out the top 5 states where the order delivery is really fast as compared to the estimated date of delivery.
You can use the difference between the averages of actual & estimated delivery date to figure out how fast the delivery was for each state.

```
with base as (select order_id, customer_id,
timestamp_diff(order_delivered_customer_date, order_purchase_timestamp, day) as
time_to_deliver,timestamp_diff(order_estimated_delivery_date,
order_purchase_timestamp, day) as estimated_time_to_deliver
from `target_business_case.orders`
```

```sql
where order_delivered_customer_date is not null),

base1 as (select c.customer_state, round(avg(b.time_to_deliver),0)
avg_time_to_deliver, round(avg(b.estimated_time_to_deliver),0) as
avg_estimated_time_to_deliver from base as b
inner join `target_business_case.customers` as c
on b.customer_id = c.customer_id
group by 1
having avg(b.time_to_deliver) < avg(b.estimated_time_to_deliver))

select * from base1 order by avg_time_to_deliver limit 5
```



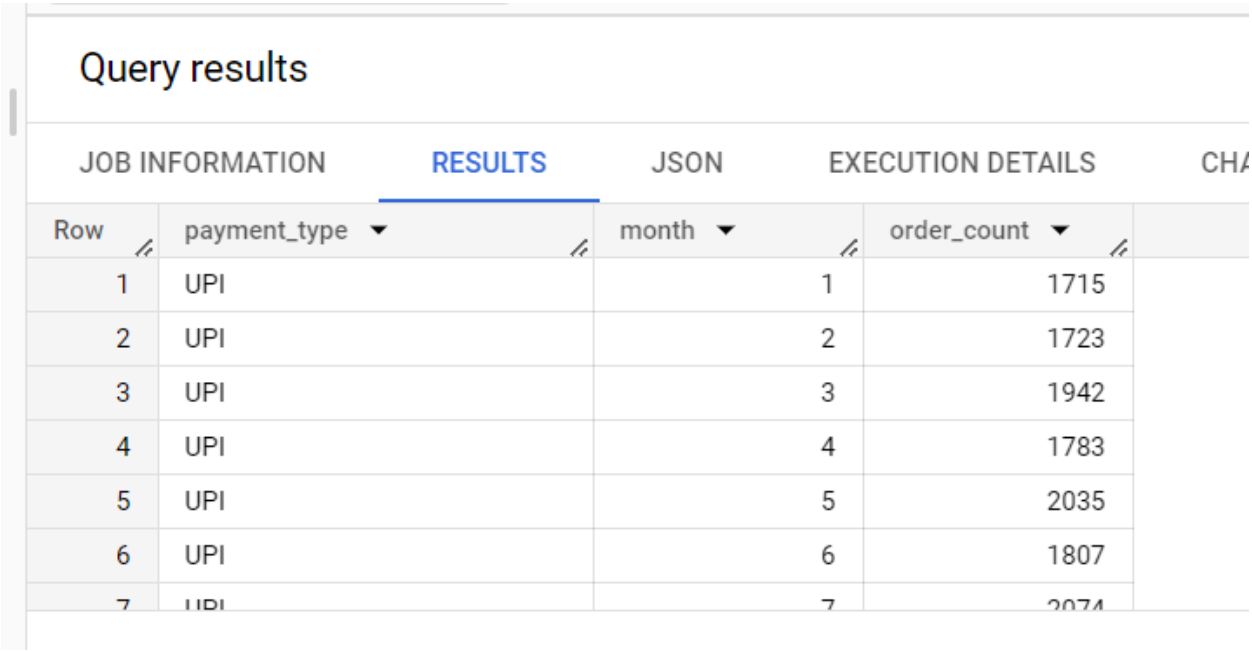
Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | CHART

Row	customer_state	avg_time_to_deliver	avg_estimated_time
1	SP	8.0	19.0
2	MG	12.0	24.0
3	PR	12.0	24.0
4	DF	13.0	24.0
5	SC	14.0	25.0

Q16. Find the month on month no. of orders placed using different payment types.

```sql
select p.payment_type,
  extract (MONTH from o.order_purchase_timestamp) as month,
  count(distinct o.order_id) as order_count
from `target_business_case.orders` o
join `target_business_case.payments` p
on o.order_id = p.order_id
group by 1, 2
order by 1, 2;
```



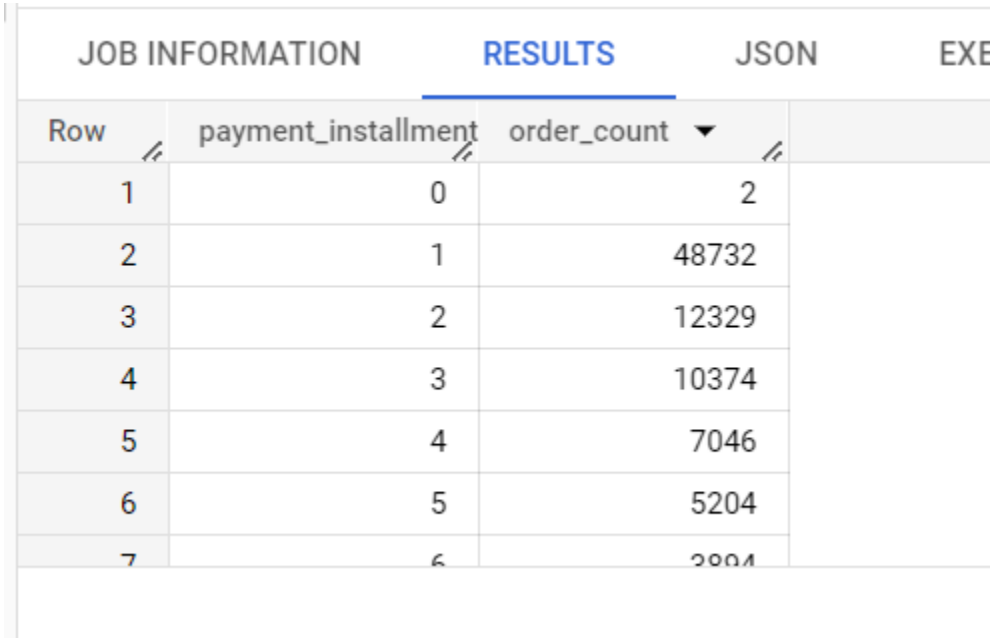
Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | CHA

Row	payment_type	month	order_count
1	UPI	1	1715
2	UPI	2	1723
3	UPI	3	1942
4	UPI	4	1783
5	UPI	5	2035
6	UPI	6	1807
7	UPI	7	2074

Q17. Find the no. of orders placed on the basis of the payment installments that have been paid.

```
select p.payment_installments,
  count(distinct o.order_id) as order_count
from `target_business_case.orders` o
join `target_business_case.payments` p
on o.order_id = p.order_id
where o.order_status <> 'canceled'
group by 1
order by 1
```



JOB INFORMATION | RESULTS | JSON | EXE

Row	payment_installment	order_count
1	0	2
2	1	48732
3	2	12329
4	3	10374
5	4	7046
6	5	5204
7	6	3894